

Introduction to Python & Setup

What is Python?

- Python is not a word that you know as a snake!!
- It is a programming language that we use in technology.
- The word “Python” stands for a specific philosophy of coding. That philosophy is officially called **The Zen of Python** (a list of 19 guiding principles written by software developer Tim Peters).
- Python is a high-level, general-purpose programming language.
 - High-level means it is designed to be easily read and written by humans. It uses plain English words instead of complex computer code.
 - General-purpose means it isn't a one-trick pony. Some languages are only used for web pages or databases. Python can build almost anything, from simple automation scripts to complex Artificial Intelligence.

So there are 19 Principles for writing computer software which are written by Tim Peters:

1. Beautiful is better than ugly.
 - a. What it means: Code should be clean, nicely spaced, and satisfying to read. Writing sloppy code just because “it works” is not the Python way.
 - b. But there's one fun fact, if code works and you don't know how, let it be.
2. Explicit is better than implicit.
 - a. What it means: Don't hide what your code is doing. It is better to write a little extra code that clearly states its purpose than to write a “clever” one-liner that looks like magic and confuses other developers.
3. Simple is better than complex.
 - a. What it means: If you can solve a problem using a basic tool (like a simple loop), don't build a massive, over-engineered system just to look smart.
4. Complex is better than complicated
 - a. What it means: If a problem is genuinely difficult and requires a complex solution, that's fine. But keep it organized. Don't let your code become complicated (a tangled mess of unreadable logic).
 - b. Show example photo.
5. Flat is better than nested.
 - a. What it means: Avoid nesting code deeply inside other code. For example, putting an **if** statement inside a **for** loop inside another **if** statement inside a function makes your eyes hurt. Keep your code structures as flat and linear as possible.
 - b. I know you don't know what is **for**, **if**, **loop**. We will find out together, the reason for this sentence is that don't make everything complicated, and sometimes, having nothing is better than having massive-enormous-messy things.
6. Sparse is better than dense.
 - a. What it means: Give your code room to breathe, although it's not alive. Give some space in your code, writing massive-dense-amount of code can sharpen your eyes. Don't jam multiple commands onto a single line. Use blank lines to separate different ideas or steps in your program, like paragraphs in a book.

7. Readability counts.
 - a. What it means: This is the ultimate law of Python. Code is read much more often than it is written. Use clear variable (You will learn later) names like `user_age` instead of cryptic short names like `ua`.
8. Special cases aren't special enough to break the rules. (Consistency and Clean Architecture)
 - a. What it means: When building a website or app, you will constantly encounter weird, one-off scenarios. For example, if a user has one single name (Nyan), then how should he fill his First Name, Middle Name, and Last Name? So you don't really need to be messy, just to fix that one weird case.
9. Although practicality beats purity.
 - a. What it means: Stick to clean coding standards and best practices. However, if you are facing a real-world situation where following the strict rules and need to submit your project on time will make the project fail. So, your code doesn't need to be perfect or flawless. Keep it organized, make it clear, and submit the project.
10. Errors should never pass silently.
 - a. What it means: If something goes wrong in your code, never hide it. Especially bugs (you will learn later what are bugs), mostly errors, dealing with errors doesn't mean that you make many mistakes. That's the life of a developer. Beginners often try to hide error messages so their project doesn't show a scary red screen. Hiding a bug doesn't fix it; it just creates a "silent killer" that will ruin your data down the line.
11. Unless explicitly silenced.
 - a. What it means: If your code breaks or encounters a bug, let it crash or explicitly log the errors so you can fix it. Never block or hide an error message just to pretend nothing went wrong. If you do decide to ignore an error, you must write code that intentionally handles it.
12. In the face of ambiguity, refuse the temptation to guess.
 - a. What it means: If your code gets confusing data or an ambiguous result, don't write code that "Guesses" what it means. Additionally, don't let your code guess something. Stop, write strict logic, or raise an error to clarify exactly what should happen.
13. There should be one, and preferably only one, obvious way to do it.
 - a. What it means: Unlike some languages where there are 10 different ways to write a simple loop, Python aims to have one standard, obvious, and clean way to do things. This ensures that if you look at another Python developer's code, you can understand instantly.
14. Although that way may not be obvious at first unless you're Dutch.
 - a. What it means: This is an inside joke! Python's creator, Guido van Rossum, is Dutch. It's a lighthearted way of saying that the "Obvious Python Way" might require a bit of learning and practice before it becomes second nature to you.
15. Now is better than never.
 - a. I hope this one is super clear for you.
16. Although never is often better than right now.
 - a. What it means: Don't get stuck in "analysis paralysis", start building your features now. However, don't rush and write unreasonable code or

complicated code to hit the deadline; take the time to do it right. Taking time is not wasting, it's to make a better one.

17. If the implementation is hard to explain, it's a bad idea.

- a. What it means: If you finish your project and you realize you can't explain how it works to another person in simple terms, the code is too complicated. It means you've created a mess. Good code is elegant and simple enough that you could easily explain the logic to a friend. If it's hard to explain, delete it and rewrite it simpler.

18. If the implementation is easy to explain, it may be a good idea.

- a. I hope that I don't really need to explain here.

19. Namespaces are one honking great idea, let's do more of those.

- a. What it means: A "namespace" is just Python's system for keeping variable names separate so they don't clash. It keeps code isolated, organized, and modular.

A quick Comparison:

To see what the "meaning" of Python looks like in practice, look at how you print text in a language like Java VS Python:

In Java:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello!");  
    }  
}
```

In Python:

```
print("Hello!")
```

Ultimately, the meaning of Python is simplicity and speed. It was designed to remove unnecessary punctuation and complicated syntax, allowing a creator to focus entirely on solving the problem at hand.

How Python Works Under the Hood

- Computers do not speak English; they only speak binary (0 and 1). Python uses a hidden translator tool called an **Interpreter**.
- When you run your code, the Python Interpreter reads your file line by line, from top to bottom, and translates your commands into machine code(binary code) instantly.

